# PYTHON DEVELOPMENT

Complete Domain Learning Guide

A practical, student-friendly guide to Python fundamentals, data structures, functions, object-oriented programming, error handling, files, libraries, testing, automation, and clean performance-aware code.

**How to use this guide**
**Read each module in order, type the code examples into a Python interpreter, predict the output before running them, and answer the checkpoints without looking at the answers. Finish with the 20-question practice assessment.**

**Assessment target: 70% — at least 14 correct answers out of 20. Retakes can be used to learn from mistakes.**

# Module 1 — Python Basics

*Syntax, types, and the interpreter's mental model*

## 1.1 How Python executes code

Python executes statements in order, from top to bottom, unless control flow changes that order. The interpreter evaluates expressions, binds names to objects, and calls functions. A variable is best understood as a name referring to an object—not as a box with a permanently fixed type. The same name can refer to an integer and later to a string.

```
score = 10
print(score)       # 10
score = "complete"
print(score)       # complete
```

**Key takeaway: The assignment score = 10 binds the name score to an integer object. The later assignment binds that same name to a string object. Python is dynamically typed, but each object still has a type.**

## 1.2 Core data types

Common built-in types include int (whole numbers), float (numbers with a fractional part), str (text), bool (True or False), NoneType (the single value None), and containers such as list, tuple, set, and dict. Use type(value) to inspect a value's type. Convert deliberately with int(), float(), str(), or bool(); conversion can fail or behave differently than expected.

```
age = int("21")
price = float("19.95")
name = "Mira"
is_active = True
missing_value = None
print(type(age).__name__)  # int
```

**Key takeaway: Input received from input() is text. If a user types 25, input() returns "25", so numeric comparisons require conversion.**

## 1.3 Control flow and comprehensions

if/elif/else selects a branch; for loops iterate over an iterable; while loops repeat while a condition is true. A list comprehension builds a new list from an iterable and may include a filter. Use it when the transformation is simple and readable; use a normal loop when logic becomes complicated.

```
numbers = [1, 2, 3, 4, 5, 6]
squares_of_evens = [n * n for n in numbers if n % 2 == 0]
print(squares_of_evens)  # [4, 16, 36]
```

**Key takeaway: Read the comprehension as: for each n in numbers, keep it if it is even, then place n*n in the new list.**

## 1.4 Tracebacks and debugging

A traceback is Python's report of the path through function calls leading to an exception. Read it from the bottom upward: the final line usually names the exception and message; the preceding frames show where execution travelled. SyntaxError means Python could not parse the code. TypeError often means an operation received an unsuitable type; NameError means a name cannot be found; IndexError means a sequence index is out of range; KeyError means a dictionary key is absent.

```
prices = [10, 20]
print(prices[2])
# IndexError: list index out of range
```

**Key takeaway: The valid indexes are 0 and 1. The traceback points to the failing line; inspect the data and index rather than blindly changing unrelated code.**

## 1.5 Mutability and object identity

Mutable objects can change in place. Lists, dictionaries, and sets are mutable; integers, floats, strings, and tuples are immutable (a tuple can still contain a mutable object). Assignment creates another name for the same object, not a copy. This explains why modifying a list through one name can be visible through another.

```
first = [1, 2]
second = first
second.append(3)
print(first)  # [1, 2, 3]

text = "cat"
text = text + "s"  # creates/binds to a new string
```

**Key takeaway: If an independent list is needed, make a copy, such as second = first.copy() for a shallow copy. Nested mutable objects may still be shared in a shallow copy.**

## Module checkpoint

**What does a variable name refer to in Python?**

Answer: An object; the name itself does not have a fixed type.

**What is the difference between a list and a tuple?**

Answer: A list is mutable; a tuple is immutable as a sequence.

**How should you begin reading a traceback?**

Answer: Start at the final exception line, then inspect the frames and failing line above it.

ROYSTIAN FOUNDATION

# Module 2 — Data Structures in Python

*Picking the right container for the job*

## 2.1 Lists

A list is an ordered, mutable sequence. It allows duplicates and supports indexing and slicing. Indexing by position is typically O(1); appending is amortized O(1); searching for a value is O(n); inserting at the beginning is O(n) because later elements must shift.

```
tasks = ["email", "review", "deploy"]
tasks.append("test")
tasks[1] = "code review"
print(tasks)
```

**Key takeaway: Use lists when order matters, items may repeat, or the collection needs to change.**

## 2.2 Dictionaries

A dictionary maps unique, hashable keys to values. It is a natural choice for records, counters, and fast lookup by identifier. Average lookup, insertion, and deletion are O(1), though worst-case behaviour can be worse. Keys must be hashable; strings, numbers, and tuples of hashable values can be keys, while lists cannot.

```
student = {"id": 104, "name": "Asha", "score": 92}
print(student["name"])
student["score"] = 95
student["passed"] = True
```

**Key takeaway: Use .get("key", default) when a missing key is expected and should not raise KeyError.**

## 2.3 Sets and tuples

A set stores unique hashable values and supports fast membership checks and operations such as union, intersection, and difference. Sets are unordered, so do not rely on display order. A tuple is an ordered, immutable sequence, useful for fixed groups of values and as a dictionary key when all its elements are hashable.

```
attended = {"Asha", "Ravi", "Mina"}
submitted = {"Ravi", "Mina", "Dev"}
print(attended & submitted)  # intersection

coordinate = (17.4, 78.5)
```

**Key takeaway: Use a set to remove duplicates or test membership. Use a tuple when a record should not be structurally changed.**

## 2.4 Nesting structures to model records

Real data often combines structures: a list of dictionaries can represent many students; a dictionary of lists can group items by category. Choose a structure based on the operations you perform most often, not just on what looks familiar.

```
students = [
    {"name": "Asha", "marks": [88, 91]},
    {"name": "Ravi", "marks": [79, 85]},
]
for student in students:
    average = sum(student["marks"]) / len(student["marks"])
    print(student["name"], average)
```

**Key takeaway: The outer list preserves student order. Each dictionary labels fields, and the marks list stores repeated scores.**

## 2.5 Time complexity

Big-O describes how resource use grows with input size, ignoring constants and lower-order terms. O(1) is constant, O(log n) grows logarithmically, O(n) grows linearly, O(n log n) is common for efficient comparison sorting, and $O(n^2)$ often appears with nested loops over the same data. Complexity is a guide, not a precise runtime prediction.

```
# O(n): one pass
for item in items:
    process(item)

# Often O(n^2): compare every item with every other item
for a in items:
    for b in items:
        compare(a, b)
```

**Key takeaway: For large collections, replacing repeated list membership checks with a set can reduce average lookup work substantially.**

## Module checkpoint

**Which structure is suitable for mapping employee IDs to records?**

Answer: A dictionary.

**Why can a set be useful for membership checks?**

Answer: It stores unique hashable values and usually offers average O(1) membership checks.

**What does O(n) communicate?**

Answer: The operation's cost grows roughly in proportion to input size.

# Module 3 — Functions & Modules

*Organising code so it can be reused and tested*

## 3.1 Functions with one clear purpose

A function groups reusable behaviour. Parameters define inputs; return sends a result back to the caller. Prefer small functions with meaningful names and a clear responsibility. print() displays information, whereas return makes a value available to other code.

```
def calculate_total(price, quantity):
    return price * quantity

total = calculate_total(12.5, 4)
print(total)  # 50.0
```

**Key takeaway: A function that returns a value is easier to test than one that only prints it.**

## 3.2 Positional, keyword, default, and variadic arguments

Positional arguments are matched by position; keyword arguments identify parameters by name. Defaults allow optional values. *args gathers extra positional arguments into a tuple; **kwargs gathers extra keyword arguments into a dictionary. Avoid mutable defaults such as [] because the same object can be reused across calls.

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Asha"))
print(greet(name="Ravi", greeting="Welcome"))

def total(*numbers):
    return sum(numbers)
```

**Key takeaway: Use keyword arguments for clarity when a function has several options. Use defaults for common cases, and variadic arguments only when a flexible number of values is genuinely useful.**

## 3.3 Pure functions and side effects

A pure function's result depends only on its inputs and it does not modify external state or perform observable side effects. Pure functions are easier to reason about, reuse, and test. Reading a file, changing a global variable, sending a message, or modifying a passed-in list are side effects.

```
def discounted(price, rate):
    return price * (1 - rate)

print(discounted(100, 0.10))  # 90.0
```

**Key takeaway: Keep calculations pure when possible, then place file, network, or user-interface work at the edges of the program.**

## 3.4 Scope

Local names exist inside a function. Parameters are local names. Global names are defined at module level; modifying a global from a function requires the global keyword, but excessive global state makes code harder to test. Python resolves names using the LEGB rule: Local, Enclosing, Global, Built-in.

```
tax_rate = 0.05

def final_price(price):
    tax = price * tax_rate  # reads a global name
    return price + tax
```

**Key takeaway: Prefer passing important dependencies as parameters rather than relying on hidden global state.**

## 3.5 Modules and packages

A module is a Python file that can be imported. A package groups modules, commonly in a directory with an __init__.py file (namespace packages also exist). Keep related functions together and use imports to reuse them. Protect script-only behaviour with if __name__ == "__main__": so importing a module does not automatically run its command-line workflow.

```
# file: math_tools.py
def double(value):
    return value * 2

# file: app.py
from math_tools import double
print(double(7))
```

**Key takeaway: Use clear module names that do not shadow standard-library modules such as json.py, random.py, or typing.py.**

## Module checkpoint

**Why is return often more reusable than print?**

Answer: The caller can store, combine, test, or transform the returned value.

**What do *args and **kwargs collect?**

Answer: *args collects extra positional arguments in a tuple; **kwargs collects extra keyword arguments in a dictionary.

**What does LEGB stand for?**

Answer: Local, Enclosing, Global, Built-in.

# Module 4 — Object-Oriented Programming

*Modelling problems with classes and behaviour*

## 4.1 Classes, objects, attributes, and methods

A class defines a kind of object; an instance is a concrete object created from the class. Attributes store state; methods implement behaviour. __init__ initialises an instance, and self refers to that instance. Each instance can hold different attribute values while sharing method definitions.

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def passed(self):
        return self.score >= 70

asha = Student("Asha", 84)
print(asha.passed())  # True
```

**Key takeaway: Use classes when data and the operations that act on that data belong together.**

## 4.2 Encapsulation

Encapsulation keeps an object's state and rules behind a clear interface. Python uses conventions rather than strict private fields: a leading underscore (such as _balance) signals internal use. Properties can validate assignments. The goal is not secrecy alone; it is preventing invalid state and reducing coupling.

```
class BankAccount:
    def __init__(self, balance=0):
        if balance < 0:
            raise ValueError("Balance cannot be negative")
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit must be positive")
        self._balance += amount
```

**Key takeaway: Expose operations such as deposit() rather than letting every caller change internal state however it likes.**

## 4.3 Inheritance and polymorphism

Inheritance models an “is-a” relationship and lets a subclass reuse or override parent behaviour. Polymorphism allows different objects to respond to the same method call in their own way. Use inheritance when the subtype truly satisfies the parent's contract; do not use it only to avoid copying a few lines.

```
class Notification:
    def send(self, message):
        raise NotImplementedError

class EmailNotification(Notification):
    def send(self, message):
        return f"Email sent: {message}"

class SMSNotification(Notification):
    def send(self, message):
        return f"SMS sent: {message}
```

**Key takeaway: Both notification types expose send(), so caller code can work with either without knowing every implementation detail.**

## 4.4 Composition and dunder methods

Composition builds a class using other objects (“has-a”) and often keeps designs flexible. For example, a Car has an Engine. Dunder methods are special methods such as __str__, __len__, __iter__, and __eq__ that integrate objects with Python's protocols.

```
class Engine:
    def start(self):
        return "engine started"

class Car:
    def __init__(self, engine):
        self.engine = engine

    def start(self):
        return self.engine.start()

car = Car(Engine())
print(car.start())
```

**Key takeaway: Composition is usually a strong default when one object collaborates with another but is not a subtype of it.**

## Module checkpoint

**What is the difference between a class and an instance?**

Answer: A class defines a kind of object; an instance is one concrete object of that class.

**When is composition often preferable?**

Answer: When an object has or collaborates with another object rather than being a subtype of it.

**What are dunder methods?**

Answer: Special methods such as __str__ and __len__ that integrate objects with Python protocols.

# Module 5 — Error Handling

*Failing safely and informatively*

## 5.1 Exceptions and raising errors

An exception interrupts normal execution. Common examples include ValueError for a value of the right general type but invalid content, TypeError for an unsupported operation or argument type, FileNotFoundError for a missing file, and ZeroDivisionError for division by zero. Use raise to report invalid conditions rather than allowing incorrect results to spread.

```
def percentage(score, maximum):
    if maximum <= 0:
        raise ValueError("maximum must be positive")
    return score / maximum * 100
```

**Key takeaway: Raise an exception that explains the violated rule and use a suitable exception type.**

## 5.2 try, except, else, finally

Put the risky operation in try. Use except to handle specific expected failures. The else block runs only if the try block completes without an exception. The finally block runs whether an exception occurred or not, so it is useful for cleanup. Catching Exception broadly can hide programming bugs; avoid bare except unless there is a carefully justified reason.

```
try:
    number = int("42")
except ValueError:
    print("Please provide a whole number.")
else:
    print("Parsed:", number)
finally:
    print("Parsing attempt finished.")
```

**Key takeaway: Keep try blocks narrow and handle errors at the level where you can respond meaningfully.**

## 5.3 Custom exceptions

A custom exception gives a domain-specific failure a distinct type. Usually subclass Exception and include it when callers need to distinguish that condition from other failures. Do not create custom exception classes for every tiny detail; create them when they improve clarity or error handling.

```
class InsufficientBalanceError(Exception):
    pass

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientBalanceError("Not enough funds")
    return balance - amount
```

**Key takeaway: A caller can catch InsufficientBalanceError without accidentally catching unrelated errors such as a typo-induced NameError.**

## 5.4 Do not swallow errors

An empty except block or a message such as “something went wrong” can make failures impossible to diagnose. Log or report enough context without exposing secrets. When re-raising after adding context, use raise or exception chaining with raise NewError(...) from err. Validate user input at boundaries and keep unexpected errors visible during development.

```
try:
    value = int(user_text)
except ValueError as err:
    raise ValueError("Age must be a whole number") from err
```

**Key takeaway: Do not catch an error unless you can recover, add useful context, or translate it into a suitable response.**

## Module checkpoint

**When does a finally block run?**

Answer: Whether or not an exception occurred, as execution leaves the try statement, barring process-level termination.

**Why catch a specific exception?**

Answer: It handles expected failures without hiding unrelated programming errors.

**Why define a custom exception?**

Answer: To give a meaningful domain-specific failure a distinct type.

# Module 6 — Files, JSON & CSV

*Reading and writing formats used in real projects*

## 6.1 Context managers and safe file access

The with statement uses a context manager to manage setup and cleanup. For files, it ensures the file is closed even if an exception occurs. Choose an explicit mode: r reads, w overwrites or creates, a appends, and x creates only if the file does not already exist. Use pathlib for convenient path handling.

```
from pathlib import Path

path = Path("notes.txt")
with path.open("w", encoding="utf-8") as file:
    file.write("Python practice\n")

with path.open("r", encoding="utf-8") as file:
    content = file.read()
```

**Key takeaway: Opening with w can erase existing contents. Use a for appending and x when overwriting must be prevented.**

## 6.2 Serialisation and JSON

Serialisation converts an in-memory structure into a format that can be stored or transmitted. JSON represents objects, arrays, strings, numbers, booleans, and null. Python's json module converts common Python structures to JSON text with dumps() and parses JSON text with loads(); dump() and load() work with file objects. JSON does not directly represent every Python object, such as sets or custom classes.

```
import json

record = {"name": "Asha", "active": True, "scores": [82, 91]}
text = json.dumps(record, indent=2)
restored = json.loads(text)
print(restored["scores"][0])  # 82
```

**Key takeaway: Malformed JSON raises json.JSONDecodeError. Validate parsed data before trusting it.**

## 6.3 CSV and malformed rows

CSV stores tabular data as rows and fields. Use Python's csv module rather than splitting each line on commas, because quoted fields can themselves contain commas or quotes. DictReader maps column names to values; DictWriter writes dictionaries. CSV values are text, so convert numeric fields explicitly and handle missing or malformed fields.

```
import csv

with open("scores.csv", newline="", encoding="utf-8") as file:
    reader = csv.DictReader(file)
    for row in reader:
        try:
            score = float(row["score"])
        except (ValueError, KeyError) as err:
            print("Skipping invalid row:", err)
            continue
        print(row["name"], score)
```

**Key takeaway: newline="" is the recommended pattern for CSV files. Decide whether invalid rows should be skipped, reported, or cause the whole import to fail.**

## 6.4 Encoding and Unicode

Text is represented in memory as characters and stored or transmitted as bytes. UTF-8 is a widely used encoding that supports characters from many writing systems. If a file is opened with a mismatched encoding, characters may be corrupted or decoding may fail. Specify encoding explicitly for portable file handling.

```
with open("messages.txt", "w", encoding="utf-8") as file:
    file.write("Namaste — ■■■■■■")
```

**Key takeaway: Use errors="replace" only when replacement is acceptable; do not silently discard meaningful data during a migration or financial import.**

## Module checkpoint

**Why use with open(...) for files?**

Answer: It ensures cleanup, including closing the file, when the block exits.

**Why use csv.DictReader instead of splitting on commas?**

Answer: It correctly handles CSV quoting and maps columns to names.

**What is UTF-8?**

Answer: A character encoding that represents Unicode text as bytes.

# Module 7 — Working With Libraries

*Using the standard library and the Python ecosystem*

## 7.1 Standard library

Python ships with many modules for common tasks: pathlib for paths, json for JSON, csv for CSV, datetime for dates and times, collections for specialised containers, math for mathematical functions, statistics for basic statistics, logging for runtime records, and argparse for command-line arguments. The standard library reduces unnecessary external dependencies.

```
from collections import Counter

words = ["python", "data", "python", "testing"]
print(Counter(words).most_common(1))
```

**Key takeaway: Read the official module documentation to understand accepted inputs, return values, exceptions, and version-specific behaviour.**

## 7.2 Virtual environments

A virtual environment isolates a project's installed packages from other projects and the system Python. This reduces version conflicts and makes the setup more reproducible. Create one with python -m venv .venv and activate it using the command appropriate for your operating system.

```
# Create an environment
python -m venv .venv

# Windows PowerShell
.venv\Scripts\Activate.ps1

# macOS/Linux
source .venv/bin/activate
```

**Key takeaway: Use the environment's Python and pip consistently. If a command installs a package globally by mistake, confirm which interpreter and package manager are active.**

## 7.3 Package managers and dependencies

pip installs packages from package indexes. A requirements.txt file can record project dependencies, and pip freeze can capture installed versions. Pinning versions improves repeatability, although a production project may also use a lock file or a tool such as Poetry or uv. Keep development and production dependencies organised, and update dependencies deliberately.

```
python -m pip install requests
python -m pip freeze > requirements.txt
python -m pip install -r requirements.txt
```

**Key takeaway: A reproducible setup needs more than code: it needs compatible Python and dependency versions and documented environment assumptions.**

## 7.4 Learning a new library

Start with the library's purpose, installation instructions, supported Python versions, quick-start example, core objects, error behaviour, and upgrade notes. Run a minimal example before integrating it into a larger project. Avoid copying code without understanding what data it accepts or what it changes.

```
import datetime

today = datetime.date.today()
print(today.isoformat())
```

**Key takeaway: Prefer maintained libraries with clear documentation and a suitable licence. Review permissions and security posture before adding a dependency.**

## Module checkpoint

**What problem does a virtual environment solve?**

Answer: It isolates dependencies so projects can use different package versions.

**What does dependency pinning improve?**

Answer: Repeatability by recording intended versions.

**Name two useful standard-library modules.**

Answer: Examples include pathlib, json, csv, datetime, logging, argparse, and collections.

# Module 8 — Writing Testable Python

*Proving that code behaves as intended*

## 8.1 Unit tests and assertions

A unit test checks one small unit of behaviour, often a function or method, in a repeatable way. A good test arranges inputs, acts, and asserts the expected outcome. Test normal cases, boundary values, and invalid inputs. Tests should be deterministic and independent where possible.

```
def is_adult(age):
    return age >= 18

def test_is_adult():
    assert is_adult(18) is True
    assert is_adult(17) is False
```

**Key takeaway: An assertion fails when the observed result differs from the expected result. A passing test suite provides evidence, not mathematical proof, of correctness.**

## 8.2 pytest and fixtures

pytest discovers test files commonly named test_*.py and test functions beginning with test_. Fixtures provide reusable setup such as sample records or temporary files. Parameterisation runs the same test logic against several input/output pairs, reducing duplication and improving edge-case coverage.

```
import pytest

@pytest.mark.parametrize(
    "value, expected",
    [(0, True), (1, False), (2, True), (7, False)]
)
def test_is_even(value, expected):
    assert (value % 2 == 0) is expected
```

**Key takeaway: Each case should be understandable on its own. Use descriptive test names that state the behaviour being verified.**

## 8.3 Mocks and test isolation

A mock replaces a real dependency so a unit can be tested without calling a network, sending an email, charging a card, or depending on external state. Mocks are useful at system boundaries, but excessive mocking can make tests verify implementation details instead of real behaviour.

```
from unittest.mock import Mock

sender = Mock()
sender.send("Welcome")
sender.send.assert_called_once_with("Welcome")
```

**Key takeaway: Mock the external dependency, not every internal function. Keep a smaller number of integration tests to confirm components work together.**

## 8.4 Test coverage and quality

Coverage measures which lines or branches tests execute. High coverage does not guarantee that assertions are meaningful or that requirements are correct. A test suite should target important behaviours, boundaries, failure modes, and regression risks—not chase a percentage alone.

```
# Example command if pytest is installed:
# python -m pytest
```

**Key takeaway: A test that executes a line but never checks its result may increase coverage without detecting a defect.**

## Module checkpoint

**What makes a unit test useful?**

Answer: It checks a specific behaviour with clear expected results, including important boundaries.

**What is a fixture?**

Answer: Reusable setup or data that prepares a test.

**Does 100% line coverage guarantee correctness?**

Answer: No; tests must assert meaningful behaviour and cover relevant cases.

# Module 9 — Automation & Scripting

*Using Python to remove repetitive work*

## 9.1 Command-line interfaces and argparse

A command-line interface accepts arguments and options when a script runs. argparse helps define those arguments, convert types, show help text, and report invalid input. This is more reliable than manually reading sys.argv for anything beyond a very small script.

```
import argparse

parser = argparse.ArgumentParser(description="Calculate a total")
parser.add_argument("price", type=float)
parser.add_argument("--quantity", type=int, default=1)
args = parser.parse_args()
print(args.price * args.quantity)
```

**Key takeaway: Running the script with --help documents how to use it. Validate ranges as well as types—for example, a quantity may need to be positive.**

## 9.2 Logging

Logging records what a script did and why. Levels commonly include DEBUG, INFO, WARNING, ERROR, and CRITICAL. Logs help diagnose scheduled jobs that run without a person watching them. Include useful identifiers and outcomes, but do not log passwords, tokens, personal data, or other secrets.

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s"
)
logging.info("Report generation started")
logging.warning("One input row was skipped")
```

**Key takeaway: Use logging instead of relying on scattered print statements in production scripts. Configure handlers and rotation for long-running jobs.**

## 9.3 Idempotent scripts and safe automation

An operation is idempotent if repeating it has the same intended effect as performing it once. Idempotency matters when jobs retry after timeouts or run on a schedule. For example, generating a report from current data and replacing the old report is usually safer than appending duplicate rows on every run. Use checkpoints, temporary output files, and explicit overwrite rules where appropriate.

```
from pathlib import Path

output = Path("summary.txt")
temporary = output.with_suffix(".tmp")

temporary.write_text("Latest summary\n", encoding="utf-8")
temporary.replace(output)
```

**Key takeaway: Think about partial failure: if the job stops halfway through, can it be safely rerun? Avoid duplicate payments, emails, or database records by using stable identifiers and idempotency controls.**

## 9.4 Scheduling and operational safety

Scripts can be scheduled with operating-system schedulers or workflow tools. A reliable scheduled task has clear inputs, a known working directory, controlled permissions, logs, failure alerts, and a predictable output location. Test with a small sample and a dry-run mode before changing many files or records.

```
A useful automation workflow:
1. Validate input paths and required fields.
2. Process a small or dry-run sample.
```

```
3. Write output to a temporary destination.
4. Validate the result.
5. Replace or publish the final output.
6. Record counts, duration, and errors.
```

**Key takeaway: Automation magnifies both benefits and mistakes. Build safeguards before scheduling a script to run unattended.**

## Module checkpoint

**Why use argparse?**

Answer: It parses, validates, and documents command-line arguments.

**What is idempotency?**

Answer: Repeating an operation has the same intended effect as doing it once.

**Why avoid logging secrets?**

Answer: Logs may be widely accessible or retained, creating a security and privacy risk.

# Module 10 — Clean Code & Performance

*Code your team can read, at the speed the job needs*

## 10.1 PEP 8 and readable design

PEP 8 describes widely used Python style conventions. Use four spaces for indentation, lowercase_with_underscores for functions and variables, CapWords for classes, and meaningful names. Keep functions focused, avoid deeply nested conditionals, and use comments to explain intent or non-obvious decisions rather than restating the code.

```
# Less clear
def f(x):
    return x * 0.9

# Clearer
def apply_ten_percent_discount(price):
    return price * 0.90
```

**Key takeaway: Consistent style lowers the effort needed to review, debug, and maintain a codebase.**

## 10.2 Generators and lazy evaluation

A generator yields values one at a time instead of building the entire result in memory. Generator functions use yield; generator expressions use parentheses. Lazy evaluation is useful when processing large files or streams. A generator is generally consumed as it is iterated, so it may not be reusable without creating it again.

```
def read_nonempty_lines(path):
    with open(path, encoding="utf-8") as file:
        for line in file:
            line = line.strip()
            if line:
                yield line

for line in read_nonempty_lines("large.txt"):
    process(line)
```

**Key takeaway: A list comprehension materialises all results; a generator can keep memory use much lower when only one item at a time is needed.**

## 10.3 Profiling before optimisation

Profiling measures where a program spends time or memory. Start with a correct, readable implementation, measure it using representative data, identify the actual bottleneck, then change and measure again. Big-O reasoning helps identify growth problems, while profiling reveals real costs such as I/O, network latency, or an unexpectedly expensive function.

```
# A simple timing experiment
from time import perf_counter

start = perf_counter()
result = sum(range(1_000_000))
elapsed = perf_counter() - start
print(f"Elapsed: {elapsed:.4f}s")
```

**Key takeaway: One timing run can be noisy. Use representative inputs and repeated measurements; for deeper analysis, explore cProfile or a dedicated benchmark tool.**

## 10.4 Premature optimisation and practical trade-offs

Premature optimisation means making code more complex for speed before evidence shows a performance problem. Optimisation can reduce readability or increase maintenance risk. Prefer clear algorithms and suitable data structures first; optimise measured hot spots while preserving correctness with tests.

```
# Repeated list membership can be expensive for large inputs
allowed_list = list(range(100_000))
allowed_set = set(allowed_list)
```

```
# Average membership lookup is typically faster in a set
is_allowed = 99_999 in allowed_set
```

**Key takeaway: Choose improvements by considering runtime, memory, readability, reliability, and the likely size of the data—not speed alone.**

## Module checkpoint

**What does a generator's yield do?**

Answer: It produces a value lazily and suspends the function until the next iteration.

**What is profiling used for?**

Answer: Measuring where time or other resources are spent.

**Why avoid premature optimisation?**

Answer: It can add complexity without improving a real bottleneck.

# Practice Assessment — 20 Questions

Choose the single best answer for each question. Give yourself one point per correct answer. A score of 14/20 (70%) meets the stated passing threshold.

**1. In Python, a variable is best described as:**

A. A permanently typed memory box

B. A name bound to an object

C. A function that returns a value

D. A special kind of list

**2. What does [n*n for n in range(5) if n % 2 == 0] produce?**

A. [0, 4, 16]

B. [0, 1, 4, 9, 16]

C. [2, 4]

D. [0, 2, 4]

**3. Which exception commonly indicates that a list index is outside valid bounds?**

A. KeyError

B. NameError

C. IndexError

D. ImportError

**4. Which data structure is usually the best fit for fast lookup by a unique employee ID?**

A. List

B. Dictionary

C. Tuple

D. String

**5. Which statement about sets is correct?**

A. They preserve duplicate values

B. They require integer elements

C. They store unique hashable elements

D. They are indexed by position

**6. Which operation is typically O(n) for a list?**

A. Access by index

B. Search for a value

C. Read its length

D. Append an item, amortized

**7. What does a function's return statement do?**

A. Always prints the value

B. Sends a result back to the caller

C. Imports a module

D. Catches exceptions

**8. What does **kwargs collect?**

A. Extra positional arguments in a list

B. Extra keyword arguments in a dictionary

C. Only integer arguments

D. Function return values

**9. Which is an example of a pure function?**

A. A function that sends an email

B. A function that changes a global counter

C. A function that returns x * 2 based only on x

D. A function that rewrites a file

**10. What is composition in OOP?**

A. Replacing all methods with variables

B. Building an object using other objects

C. Always inheriting from two parents

D. Converting a class to a module

**11. Which block runs whether or not an exception occurred?**

A. else

B. except

C. finally

D. try only

**12. Which is the best error-handling practice?**

A. Catch every exception and ignore it

B. Use specific handlers for expected failures

C. Delete all error messages

D. Use exceptions for every normal branch

**13. Why is the with statement useful for file handling?**

A. It makes all files JSON

B. It automatically sorts file content

C. It manages cleanup such as closing the file

D. It prevents all I/O errors

**14. Which module should be preferred for robust CSV parsing?**

A. csv

B. random

C. turtle

D. fractions

**15. What does a virtual environment isolate?**

A. Python syntax

B. Project dependencies

C. CPU cores

D. The operating system

**16. What does a test fixture provide?**

A. A production server

B. Reusable setup or test data

C. A guarantee of zero defects

D. A replacement for assertions

**17. What is the main benefit of mocking an external service in a unit test?**

A. It proves the real service is available

B. It makes tests depend on the internet

C. It isolates the unit from the real external dependency

D. It eliminates the need for assertions

**18. What makes an operation idempotent?**

A. It runs only once

B. Repeating it has the same intended effect as one run

C. It always runs faster the second time

D. It never writes files

**19. What is the main role of argparse?**

A. Create database tables

B. Parse command-line arguments

C. Draw charts

D. Encrypt every variable

**20. Why profile code before optimising it?**

A. To avoid writing tests

B. To identify actual bottlenecks with evidence

C. To make all code shorter

D. To remove all loops

ROYSTIAN FOUNDATION

# Answer Key with Explanations

**1.** Correct answer: **B** — Python names refer to objects; a name can later be rebound to an object of another type.

**2.** Correct answer: **A** — The filter keeps 0, 2, and 4, and the expression squares each kept value.

**3.** Correct answer: **C** — IndexError is raised when a sequence index is out of range.

**4.** Correct answer: **B** — A dictionary maps keys to values and offers average O(1) lookup.

**5.** Correct answer: **C** — Sets hold unique hashable elements and are not position-indexed sequences.

**6.** Correct answer: **B** — A linear search may inspect every item.

**7.** Correct answer: **B** — Return makes a value available to the calling code.

**8.** Correct answer: **B** — The double-star parameter collects extra keyword arguments in a dict.

**9.** Correct answer: **C** — Its result depends only on its input and it has no side effects.

**10.** Correct answer: **B** — Composition represents a has-a relationship between collaborating objects.

**11.** Correct answer: **C** — finally is intended for cleanup that should occur as control leaves the try statement.

**12.** Correct answer: **B** — Specific handlers avoid concealing unrelated bugs and make recovery more meaningful.

**13.** Correct answer: **C** — The context manager releases the file resource when the block exits.

**14.** Correct answer: **A** — The csv module handles quoting and delimiters correctly.

**15.** Correct answer: **B** — It keeps project package installations separate from other environments.

**16.** Correct answer: **B** — Fixtures prepare repeatable state or dependencies for tests.

**17.** Correct answer: **C** — A mock avoids real network calls and lets the test verify interactions or behaviour.

**18.** Correct answer: **B** — Idempotency makes retries safer and reduces duplicate side effects.

**19.** Correct answer: **B** — argparse defines options, converts values, and generates help output.

**20.** Correct answer: **B** — Profiling directs effort toward measured performance costs instead of guesses.

# Quick Revision Sheet

**Names and types:** Names refer to objects. Use type() to inspect types; convert input text deliberately.

**Core containers:** list = ordered and mutable; tuple = ordered and immutable; set = unique values and fast membership; dict = key-value mapping.

**Complexity:** List search is O(n); dictionary/set membership is average O(1); nested loops often suggest $O(n^2)$.

**Functions:** Prefer clear inputs and return values. *args collects positional extras; **kwargs collects keyword extras. Avoid mutable default arguments.

**OOP:** Class = blueprint; instance = object; encapsulation protects invariants; inheritance is is-a; composition is has-a.

**Exceptions:** Catch specific expected errors. else runs if try succeeds; finally is for cleanup. Do not swallow errors silently.

**Files and data:** Use with; specify encoding='utf-8'; use json for JSON and csv for CSV; validate parsed data.

**Libraries:** Use virtual environments, explicit dependency files, and documentation.

**Testing:** Test normal, boundary, and invalid cases. Fixtures prepare tests; mocks isolate external dependencies; coverage is not correctness.

**Automation:** Use argparse for CLI options, logging for diagnostics, and idempotent designs for safe retries.

**Performance:** Follow PEP 8, use generators for lazy processing, profile real workloads, and avoid premature optimisation.

**End of guide. Revisit any missed assessment question by reading the related module and writing a small code example of your own.**